COMP9001
Lecture 2
Variables and Data Types

Presenter: Dr Armin Chitizadeh

Version 1.0

THE UNIVERSITY OF SYDNEY

Inspired by Hazem El-Alfy 2024 slides

CRICOS 00026A TEQSA PRV12057

We recognise and pay respect to the Elders and communities – past, present, and emerging – of the lands that the University of Sydney's campuses stand on. For thousands of years they have shared and exchanged knowledges across innumerable generations for the benefit of all.

THE UNIVERSITY OF SYDNEY

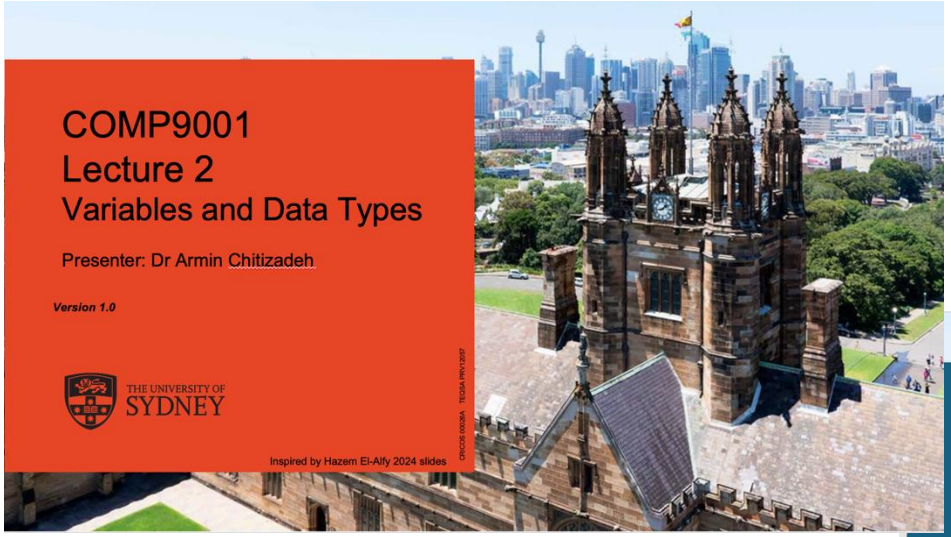
Join at menti.com | use code 6466 2498

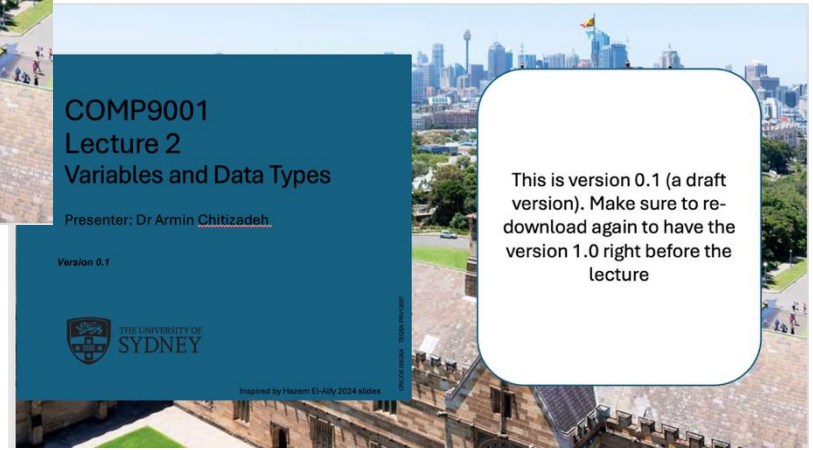
How are you?

https://www.menti.com/al4n4abxijp6

End presentation

What is with Orange and Blue versions?





Where are we in the unit?

Week 1
Intro

Week 2
Variables

Week 3
Conditionals

Week 4
Iteration

Week 5
Flow/Functions

Week 6
Containers

Week 7
Classes/Objects

Week 8
File I/O

Week 9
Adv Flow

Week 10
Testing

Week 11
Recursion

Week 12
Multi Arrays

Week 13
Revision

Final Project

In Life, Details Change

Tailor Story

Tailor Story

How He Keeps All the Records

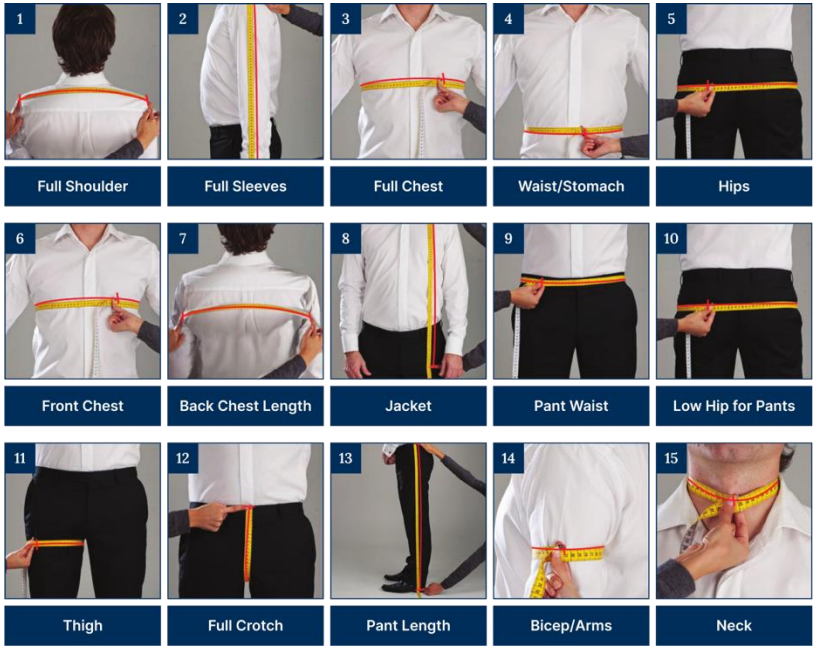
Photo by Kelly Sikkema from unsplash

My Tailor Wants to Use Python to Simplify his Life

Tailor part1

Tailor Story tailor1.py
What do they measure?

Hips: 100cm
Pant Waist: 95cm
Thigh: 25cm
Pant Length: 110cm




```python
#tailor 1
print("details of Armin Chitizadeh with customer ID of 3322445")
print("Customer Name:", "Armin Chitizadeh")
print("Customer ID:", "3322445")
print("Hips: ", "100")
print("Pant Waist: ", "95")
print("Thigh: ", "25", sep=" ")
print("Pant Length: ", "110", sep=" ")

print("Total clothes fabric for Hips of 100, Pant Waits of 95,")
print("Thigh of 25 and Pant Length of 110 is 5 meters")
```

```
achi0646@AHH6CFCMWFJ Lecture1 % python3 tailor1.py
details of Armin Chitizadeh with customer ID of 3322445
Customer Name: Armin Chitizadeh
Customer ID: 3322445
Hips:  100
Pant Waist:  95
Thigh:  25
Pant Length:  110
Total clothes fabric for Hips of 100, Pant Waits of 95,
Thigh of 25 and Pant Length of 110 is 5 meters
```

Tailor Story
Details Change

Hips: 120cm
Pant Waist: 100cm
Thigh: 28cm
Pant Length: 139cm

Variables

- Variables contain data. So, you can call them later. This makes data reusable, recallable, re-assignable etc.

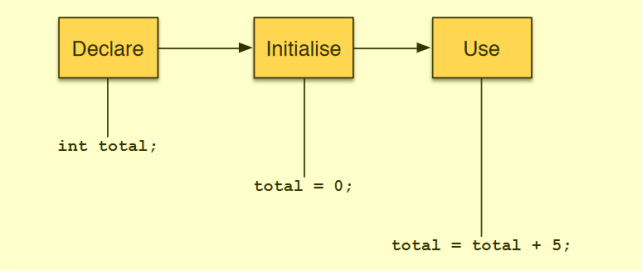
- Simplest definition of a variable is a "**named container**" of data you'd like to refer to in your program. There are 2 main reasons you may want to do this.
    1. Data usually tends to be bigger than a few characters.
       Imagine calling books by their whole content (possibly 100s of pages) each time you refer to them instead of their names.
    2. You will want to refer to your data multiple times in a program.
       So, it helps to have it in a convenient container and call it whenever we need to such as when we want to print it.

- *Demo!* Let's go to ED to create our first variables!

Variables in Programs

- Before you can use a variable, you must first declare it and **initialize** it

- This is important for static-typed languages e.g. C. Once a variable of specific type is declared, it cannot be assigned a different data type.



- In dynamically-typed languages e.g. Python, data type is determined at run-time. Programmers can re-assign a variable to a different data type after it is initialized.

Data Types and Objects

Data Types

- A set of values and a set of operations defined on those values.
- Operators are representation of a data-type operation. Can be unary, binary and ternary.
- Primitive types are basic data types that uses fixed amount of memory. Examples of these are usually: `int`, `float`, `str`, `bool`.

Object

- *Objects* are blocks of memory that contain related data and associated operations for that data.
- One object can contain many other Objects. They can have customised operators defined by the *programmer i.e., you* define the actions of operators (more later on through the semester).
- In Python, *everything* is an Object!

Variables and Data Types in Programs

- Good programmers always:
  - Know the data type that they are working with throughout the program.
  - Know the scope of each variable.
  - Initialize the variables to something useful.

- Facts we know so far:
  - Machine code is usually in **binary** format.
  - Python is a high-level language, close to **human language.** Has **four** basic data types:

type	*set of values*	*common operators*	*sample literals*
`int`	*integers*	`+ - * // % **`	`99 12 2147483647`
`float`	*floating-point numbers*	`+ - * / **`	`3.14 2.5 6.022e23`
`bool`	*true-false values*	`and or not`	`True False`
`str`	*sequences of characters*	`+`	`'AB' 'Hello' '2.5'`

Basic built-in data types

Image credit:
https://introcs.cs.princeton.edu/python/12types/

Important: Variables Names

- Must start with a letter (cannot start with numbers).
- Must not have spaces.
- Case sensitive! `X` and `x` refers two different variables.
- Avoid reserved keywords – words that have special purpose.
- Name variables to describe purpose of value.
- All CAPS indicate constants e.g. `FLAT_RATE`
- Good practice to use lowercase with words separated by underscores (_) to improve readability.
- Avoid using variable names that already exist as other objects in Python because they are already attached to existing Python functions e.g. `print, int, str.`

Important: Variables Names

Your task:

- Read: https://www.python.org/dev/peps/pep-0008/#function-and-variable-names
- Reflect: Are the contents on this slide consistent with the recommendations ?

- Must start with a letter (cannot start with numbers).
- Must not have spaces.
- Case sensitive! `X` and `x` refers two different variables.
- Avoid reserved keywords – words that have special purpose.
- Name variables to describe purpose of value.
- All CAPS indicate constants e.g. `FLAT_RATE`
- Good practice to use lowercase with words separated by underscores (_) to improve readability.
- Avoid using variable names that already exist as other objects in Python because they are already attached to existing Python functions e.g. `print, int, str.`

Reserve Keywords

```
False       class       finally     is          return
None        continue    for         lambda      try
True        def         from        nonlocal    while
and         del         global      not         with
as          elif        if          or          yield
assert      else        import      pass
break       except      in          raise
```

- Do not need to memorize.
- Most development environments will highlight these keywords in a different colour.

Practice Exercise

- Which of these identifiers are valid variable names?
  - `num`
  - `5th_person`
  - `mass_obj2`
  - `\tallest`
  - `_C`
  - `list`

- Go to Ed Lesson to check your answer.

Type Annotation

- Python does not require you to declare the type of variable during initialization because it is a dynamically-typed language.

- The type is determined at runtime. However, type annotation can be used to indicate the variable type or type of data that the variable contains.
  Example:
  ```
  x: int = 10
  ```

- In other languages e.g. C language, you must declare the type during initialization. If the value is different from the variable type, an error will be raised.

Activity: Data IRL

Identify the data type most suitable to represent the following information:

- Blood Group
- Number of cars on the road
- Your current weight
- Price of oranges e.g. $1.50/orange
- Number of siblings
- Name of your city
- Is it raining?
- Do you have an umbrella with you now?

Write Python statements to initialize variables with examples of real life values for each information. Go to: Ed Lesson Part 1

Tailor Story (tailor2.py)
In Life, Details Change

- How can we fix the issue of changing values?

tailor2.py

```python
customer_name = "Armin Chitizadeh"
customer_id = "3322445"
hips_size = 120
pant_size = 100
thigh = 28
pant = 139
print("details of", customer_name, "with customer ID of", customer_id)
print("Customer Name:", customer_name, sep=" ")
print("Customer ID:", customer_id, sep=" ")
print("Hips: ", hips_size, sep=" ")
print("Pant Waist: ", pant_size, sep=" ")
print("Thigh: ", thigh, sep=" ")
print("Pant Length: ", pant, sep=" ")
```

```
achi0646@AHH6CFCMWFJ wk02 % python3 tailor2.py
details of Armin Chitizadeh with customer ID of 3322445
Customer Name: Armin Chitizadeh
Customer ID: 3322445
Hips:  120
Pant Waist:  100
Thigh:  28
Pant Length:  139
```

Data Type - `float`

- Float variables can be initialized using a sequence of digits with decimal points or in scientific notation.

- Arithmetic operators (`+, -, *, /`) work with `float` objects too BUT results are stored as `float` objects due to implicit type casting.

- Typically have 15-17 decimal digits precision.

- Floating-point numbers are approximations of the real-numbers. Be careful when performing comparisons with floating-point numbers.

$6.02 * 10^{23}$

```
pi = 3.14159
random_float = 6.02e23
```

```
>>> 5.0 + 3.0
8.0
>>> 5.0 - 3.0
2.0
>>> 5.0 * 3.0
15.0
>>> 5.0 / 3.0
1.6666666666666667
```

Type Casting

- Conversion between data types.

- Explicit conversion – invoke using function calls.
  - `str()` → convert to type `str`
  - `int()` → convert into type `int`
  - `float()` → convert into type `float`

- Implicit conversion – Python does the conversion automatically when appropriate.
  - Example cases where results are implicitly converted to `float`:
    - Division between a `float` and `int`
    - Division between two `int`
    - Addition between an `int` and `float`

Conundrum : `int` or `float`

- Identify the information that is going to be stored and how it changes over time.

- Are they whole numbers or can there be fractional components?
  - Accuracy: Integers are exact but floating-point numbers are not.
  - Range: If you are dealing with very large or small numbers, you have to use `float` – their range is much bigger.

- Is there any danger of using a floating point type in your program if only integers are expected?

Tailor part2

Tailor Story
Sometimes We Need Calculations

How much fabric is needed?

```python
customer_name = "Armin Chitizadeh"
customer_id = "3322445"
hips_size = 120
pant_size = 100
thigh = 28
pant = 139
print("details of", customer_name, "with customer ID of", customer_id)
print("Customer Name:", customer_name, sep=" ")
print("Customer ID:", customer_id, sep=" ")
print("Hips:", hips_size, sep=" ")
print("Pant Waist:", pant_size, sep=" ")
print("Thigh:", thigh, sep=" ")
print("Pant Length:", pant, sep=" ")

print("Total clothes fabric for Hips of ", hips_size, ", Pant Waits of ", pant_si
print("Thigh of ", thigh, "and Pant Length of", pant, " is 5 meters")
```

Photo by Tima Miroshnichenko
from pexels

Expression

- An expression is a combination of literals, variables, and operators that Python evaluates to produce a value.

- Many expressions looks like mathematical formulas:

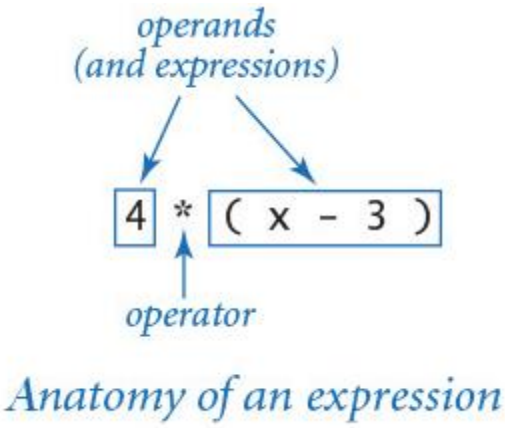


Image credit: https://introcs.cs.princeton.edu/python/12types/

How much fabric is needed?

tailor3.py

```python
customer_name = "Armin Chitizadeh"
customer_id = "3322445"
hips_size = 120
pant_size = 100
thigh = 28
pant = 139
total_fabric = ((hips_size + pant_size) * (thigh + pant))/10
print("details of", customer_name, "with customer ID of", customer_id)
print("Customer Name:", customer_name)
print("Customer ID:", customer_id)
print("Hips: ", hips_size)
print("Pant Waist: ", pant_size)
print("Thigh: ", thigh)
print("Pant Length: ", pant)
print("total fabric: ", total_fabric)
```

```
achi0646@AHH6CFCMWFJ wk02 % python3 tailor3.py
details of Armin Chitizadeh with customer ID of 3322445
Customer Name: Armin Chitizadeh
Customer ID: 3322445
Hips:  120
Pant Waist:  100
Thigh:  28
Pant Length:  139
total fabric:  367.4
```

Photo by Tima Miroshnichenko
from pexels

Activity: Data IRL

- Extend the program to calculate the total amount that you have to pay if you are to buy one orange for each sibling *and yourself.*

- Display this amount to the terminal and check if the calculation is correct using a digital/physical calculator.

- Go to: Ed Lesson Part

Reflect: What did we just do?

- Did you manage to create an expression using the variables that you have created?
- What data type are the operands for each operator?
- What data type is the number of oranges bought?
- Check your answers: Ed Lesson Part 2

Activity: Data IRL

- If the price per orange is now $1.30 and you are asked to buy 3 oranges. How much is the total amount that you have to pay to buy the oranges?

- Write codes to perform this calculation in your program.

- Once again, display the price to the terminal and check if the calculation is correct.

- Go to: Ed Lesson Part 3

Reflect: What did we just do?

- Did the program produce the correct answer?

- True/False: Floating points are accurate representations of real numbers!

- Check your answers: Ed Lesson Part 3

Tailor part3

Who Has Paid Who Has Not

Data Type - `bool`

- Two possible values – `False` (0) and `True` (1).
- Comparison operators are mixed operators that will result in a Boolean.
-

op	*meaning*	True	False
==	*equal*	2 == 2	2 == 3
!=	*not equal*	3 != 2	2 != 2
<	*less than*	2 < 13	2 < 2
<=	*less than or equal*	2 <= 2	3 <= 2
>	*greater than*	13 > 2	2 > 13
>=	*greater than or equal*	3 >= 2	2 >= 3

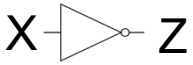
Comparisons with `int` operands and a `bool` result

- Boolean operators in Python: `not`, `and`, `or`

Truth Table

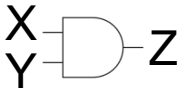
- All possible inputs and output combinations for each operation can be looked up in the truth table:

not



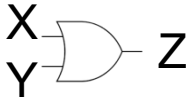
X	Z
0	1
1	0

and



X	Y	Z
0	0	0
0	1	0
1	0	0
1	1	1

or



X	Y	Z
0	0	0
0	1	1
1	0	1
1	1	1

Activity: Data IRL (part1)

- Write an expression to determine if you will get wet in the rain.
- Save the result of the expression into a variable called `is_wet`. The variable should contain only two values to represent the state: wet (True) and not wet (False).
- **Hint:** Is it raining? Does he have an umbrella?

- Go to: Ed Lesson Part 4

Activity (part2)

```
is_raining = Fasle
have_umbrella = True

is_wet = False
```

```
is_raining = True
have_umbrella = False

is_wet = True
```

```
is_raining = False
have_umbrella = False

is_wet = False
```

```
is_raining = True
have_umbrella = True

is_wet = False
```

Truth Table - Example

is_raining	has_umbrella	???	is_wet (Expected)
0	0		0
0	1		0
1	0		1
1	1		0

Note:
0 represents False
1 represents True

Activity

```
is_raining = Fasle
have_umbrella = True

is_wet = False
```

```
is_raining = True
have_umbrella = False

is_wet = True
```

```
is_raining = False
have_umbrella = False

is_wet = False
```

```
is_raining = True
have_umbrella = True

is_wet = False
```

```
# Part 4
is_wet = is_raining and not(have_umbrella)
print(is_wet)
```

Reflect: What did we just do?

- Which Boolean operator did you use?

- How did you verify that the expression is correct?

- Check your answers: Ed Lesson Part 4

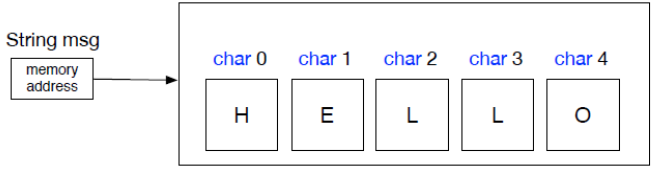
Data Type - `str`

- Observations from Lab 1:
  - Strings must be enclosed with single/double quotes.
  - Backslash character in string denotes escape character to indicate characters with special meaning e.g. `'\\'` means backslash.
  - String concatenation using `+` operator
  - String repetition using `*` operator

Data Type - `str`

- A string of characters only make sense when they are together.
- A string variable contains the memory address that <u>contains all characters</u>.

object



- Can perform *many more* operations on string variables using string methods.
  - https://docs.python.org/3/library/stdtypes.html#string-methods
  - `str.format()` returns a copy where each replacement field (placeholder {}) is replaced with a string value of the corresponding argument.
  - `str.lower()` returns a copy of `str` converted to all lowercase.
  - `str.capitalize()` returns a copy of `str` with first character capitalize.
  - `str.upper()` returns a copy of `str` converted to all uppercase.

Summary part1

- Write programs that contains variables with different data types and expressions.
- Our focus this part – integer, float and boolean.
- Next part – string methods and getting input from users.

General Advice

Let’s Play Menti!

Identify the data type most suitable to represent the following information.

https://www.menti.com/alr3vwcc38pg

Part2

Interactive program

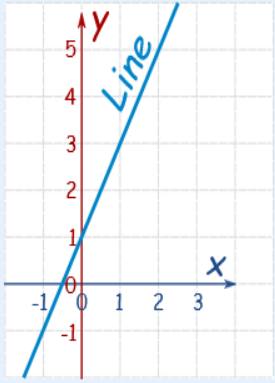
Created by Designer through Copilot

Tailor part4

Tailor Story
Sometimes We Need Input

How much will it cost?

Example: **y = 2x + 1** is a linear equation:

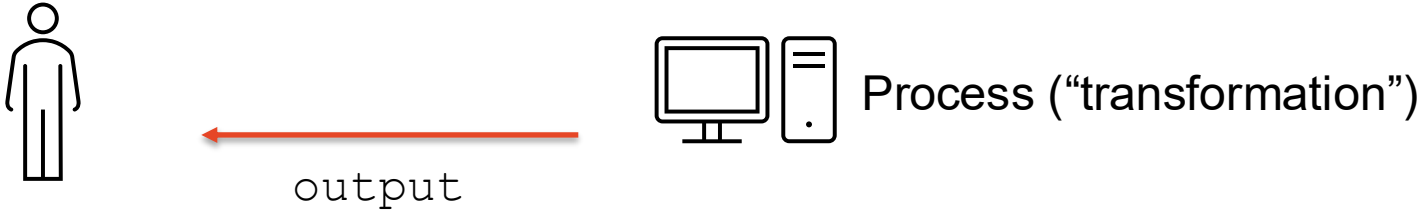


The graph of **y = 2x+1** is a straight line

Credit: https://www.mathsisfun.com/algebra/linear-equations.html

Interactive Programs

- So far



Activity: Sandwich Maker

- What is the output of the program `sandwich.py`?

sandwich.py

```
'''
Author: John Smith
This program outputs different types of sandwiches.
'''

print("1 Nutella Sandwich")
print('Order by', 'John.')
print("John, your Nutella Sandwich is ready!")
# End of program
```

- If you run the program 5 times, would the output change?
- Code readability. What do you think?
- Ed Lesson

Why are comments needed?

Your code needs to be read by another people

You will be another person after a while

2015

2019

Recap: `print()` function

- `print()` function allows concatenation of different pieces of information with a single space as separation (keyword argument – `sep`).

`print`(*`*objects, sep=' ', end='\n', file=sys.stdout, flush=False`*)

Print *objects* to the text stream *file*, separated by *sep* and followed by *end*. *sep*, *end*, *file*, and *flush*, if present, must be given as keyword arguments.

All non-keyword arguments are converted to strings like `str()` does and written to the stream, separated by *sep* and followed by *end*. Both *sep* and *end* must be strings; they can also be `None`, which means to use the default values. If no *objects* are given, `print()` will just write *end*.

The *file* argument must be an object with a `write(string)` method; if it is not present or `None`, `sys.stdout` will be used. Since printed arguments are converted to text strings, `print()` cannot be used with binary mode file objects. For these, use `file.write(...)` instead.

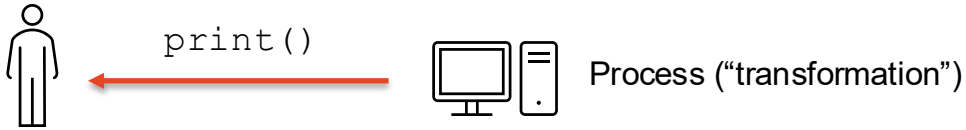
Whether the output is buffered is usually determined by *file*, but if the *flush* keyword argument is true, the stream is forcibly flushed.

Changed in version 3.3: Added the *flush* keyword argument.

Source: https://docs.python.org/3/library/functions.html#print

Interactive Programs

- Reality: Our programs so far…



- We need a way to get the input from the user!
- Simplest form: *Text* from standard *input* (keyboard)

Interactive Programs

- Programs are usually interactive.

- The `input()` function reads a line from standard input (keyboard) converts it to a **string** stripping the **newline character**

Activity: Echo – Can you hear me?

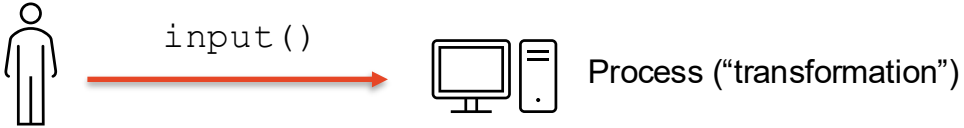
- Go to: ED Part1
- Run the program `demo.py`.

- Type something in the terminal. Press `Enter` key once you are done.

- What do you see?

What did we just do?

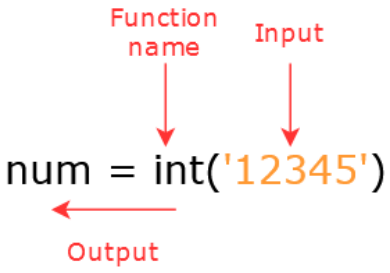
- The computer did exactly as you told it to do!



- The `input()` function reads a line from standard input (keyboard) converts it to a **string** stripping the **newline character.**
- To view the data entered by the user, use `print()` to output to the terminal.

Activity: Echo – cant you hear me?

- Go to: Ed
- Do **Part 2: Echo**
- Hint:
  - The output of `input()` is the input of `print()`.



Interactive Programs in Reality

- Data received from the user, output of the `input()` function, is usually processed e.g. compute, transform.

- Data is stored into variables for easier retrieval and manipulation.

Tailor Story
How much will it cost?

Depends on the exchange rate!

`input()` function

`input`([*prompt*])

If the *prompt* argument is present, it is written to standard output without a trailing newline. The function then reads a line from input, converts it to a string (stripping a trailing newline), and returns that. When EOF is read, `EOFError` is raised. Example:

```
>>> s = input('--> ')
--> Monty Python's Flying Circus
>>> s
"Monty Python's Flying Circus"
```

Source: https://docs.python.org/3/library/functions.html#input

- Example in documentation uses an assignment statement:
  - Variable name: ???
  - Value: ???
  - Data type: ???

How much will it cost?

Depends on the exchange rate!

tailor4.py

```python
customer_name = "Armin Chitizadeh"
customer_id = "3322445"
hips_size = 120
pant_size = 100
thigh = 28
pant = 139
total_fabric = ((hips_size + pant_size) * (thigh + pant))/10
FIXED_COST = 40
exchange_rate = input("Please give me the exchange rate Baba Yaga: ")

print("details of", customer_name, "with customer ID of", customer_id)
print("Customer Name:", customer_name)
print("Customer ID:", customer_id)
print("Hips: ", hips_size)
print("Pant Waist: ", pant_size)
print("Thigh: ", thigh)
print("Pant Length: ", pant)
print("Total fabric: ", total_fabric)

total_cost = int((40 + total_fabric/100) * int(exchange_rate))

print("Total cost: ", total_cost, "Toman")
```

```
achi0646@AHH6CFCMWFJ wk02 % python3 tailor4.py
Please give me the exchange rate Baba Yaga: 51000
details of Armin Chitizadeh with customer ID of 3322445
Customer Name: Armin Chitizadeh
Customer ID: 3322445
Hips:  120
Pant Waist:  100
Thigh:  28
Pant Length:  139
Total fabric:  3674.0
Total cost:  3913740 Toman
achi0646@AHH6CFCMWFJ wk02 %
```

Activity: Echo – Can you hear me?

- Go to: Ed
- Do **Part 3: Echo with lower volume**
- Hint:
  - String objects have many methods including `str.upper()` and `str.lower()`
  - Test these in the Python Shell:
    - `"help".upper()`
    - `"HELP".lower()`
    - `input().swapcase()`

Tailor part5

Tailor Story
I Want to Print a Table

Something like this ->

SALES			
purchase_number	date_of_purchase	customer_id	item_code
1	03/09/2016	1	A_1
2	02/12/2016	2	C_1
3	15/04/2017	3	D_1
4	24/05/2017	1	B_2
5	25/05/2017	4	B_2
6	06/06/2017	2	B_1
7	10/06/2017	4	A_2
8	13/06/2017	3	C_1
9	20/07/2017	1	A_1
10	11/08/2017	2	B_1

Output Formatting

- Many ways to present the output of a program in a more readable format.

- Two approaches:
  1. Formatted String Literals (f-strings)
     - Prefixing the string with `f` or `F`
     - Expression included inside string as `{expression}`

  2. String `format()` method
     - Objects passed into the `str.format()` replace items in brackets.

Your task:
- Run `demo.py` in Ed lesson Sandwich Maker 2.0
- Can you spot both approaches?

Output Formatting

- Two approaches:
  1. Formatted String Literals (f-strings)
     - Prefixing the string with `f` or `F`
     - Expression included inside string as `{expression}`
     - Example: Line 12
  2. String `format()` method
     - Objects passed into the `str.format()` replace items in brackets.
     - Example: Line 13

Documentation: https://docs.python.org/3/tutorial/inputoutput.html#formatted-string-literals

Format Specification

- Specify presentation format within placeholder.
- Example:
  - Width of 3 and pad with zeros e.g. `001`
  - `print(f"{quantity:03d}")`
- Actual value is <u>not</u> modified during formatting.
- Documentation: https://docs.python.org/3/library/string.html#formatspec

Tailor Story
I Want to Print a Table

Something like this ->

SALES			
purchase_number	date_of_purchase	customer_id	item_code
1	03/09/2016	1	A_1
2	02/12/2016	2	C_1
3	15/04/2017	3	D_1
4	24/05/2017	1	B_2
5	25/05/2017	4	B_2
6	06/06/2017	2	B_1
7	10/06/2017	4	A_2
8	13/06/2017	3	C_1
9	20/07/2017	1	A_1
10	11/08/2017	2	B_1

```python
# tailor5.py
customer_name = "Armin Chitizadeh"
customer_id = "3322445"
hips_size = 120
pant_size = 1004
thigh = 28
pant = 139
total_fabric = ((hips_size + pant_size) * (thigh + pant))/10
FIXED_COST = 40
exchange_rate = input("Please give me the exchange rate Baba Yaga: ")

print("  Customer Name", "Customer ID", "Hips", "Pant Waist", "Thigh", "Pant Length", sep="   ")
print(f"{customer_name}|   {customer_id}   |{hips_size:^5d}|{pant_size:^14d}|{thigh:^7d}|{pant:^12d}")

print("Total fabric: ", total_fabric)

total_cost = int((40 + total_fabric/100) * int(exchange_rate))
print(f"total_cost: {total_cost:,d} Toman")
```

Activity: Sandwich Maker 20.0

- Write a program that prompts the customer for their name, sandwich type and quantity. The program will always serve 2 extra sandwiches with their order.

- The program must produce the exact output as specified:
  - Customer name should be made capital case e.g. Naz, Sue, Maryam.
  - Sandwich type should be made all upper case.
  - The program confirms the number of sandwich by displaying all details to the terminal, see Sample Program Output.

- Ed Lesson Part 2

Summary part2

- A program usually processes data, e.g. can be from standard input using `input()`.
- Variables are used to associate a value to an identifier i.e. variable name.
- Many string methods available in Python.
- Different ways to perform output formatting using `str.format()` and f-string approach.

Self-Study –Part1

- **Identifiers and variables.**
  Chapter 2.4. – 2.5, Liang, Y. D. (2022). *Introduction to Python Programming and Data Structures*. Pearson.
- **Numeric data types and operators.**
  Chapter 2.8, Liang, Y. D. (2022). *Introduction to Python Programming and Data Structures*. Pearson.
- **Boolean Types, Values and Expressions.**
  Chapter 3.2, Liang, Y. D. (2022). *Introduction to Python Programming and Data Structures*. Pearson.
- **String Methods.**
  Chapter 4.6, Liang, Y. D. (2022). *Introduction to Python Programming and Data Structures*. Pearson.

- **Chapter 5 – Keyboard Input.** Downey, A. B. (2015). Think Python: How to Think Like a Computer Scientist (2e ed.). O’Reilly Media, Incorporated.

- **Chapter 2.** Downey, A. B. (2015). Think Python: How to Think Like a Computer Scientist (2e ed.). O’Reilly Media, Incorporated.

- **Chapter 2.3, 4.2, 4.3.** Liang, Y. (2022). Introduction to Python Programming and Data Structures, Global Edition. Pearson Education Limited.

Fully anonymous feedback form for COMP9001 students – Fill out form